

DOCUMENTACIÓN TÉCNICA COMPLETA

Gestión de Alimentos con Room Database

Proyecto de Desarrollo Android

Autor: José Carlos Sancho Acosta

Tecnologías: Kotlin, Jetpack Compose, Room Database, MVVM Architecture

1. INTRODUCCIÓN

1.1 Descripción del Proyecto

Esta aplicación Android permite gestionar **Componentes Dieta** e **Ingredientes** de forma completa, implementando operaciones CRUD (Create, Read, Update, Delete) con persistencia de datos mediante Room Database. El proyecto es una evolución del Proyecto 1, donde se migró desde persistencia en ficheros a una base de datos relacional.

3. ARQUITECTURA DE LA APLICACIÓN

3.1 Patrón MVVM (Model-View-ViewModel)

La aplicación sigue el patrón arquitectónico MVVM recomendado por Google para aplicaciones Android:

3.2 Arquitectura por Capas

Capa de Presentación (UI Layer)

- **Composables:** Funciones que definen la interfaz
- **Navigation:** Gestión de rutas entre pantallas
- **States:** Estados de UI reactivos

Capa de Dominio (Domain Layer)

- **ViewModels:** Gestión de lógica de negocio

- **StateFlow:** Emisión de estados observables
- **Use Cases:** Operaciones específicas

Capa de Datos (Data Layer)

- **BDController:** Patrón Repository
- **Room Database:** Base de datos local
- **DAOs:** Acceso a datos
- **Entities:** Modelos de datos

4. TECNOLOGÍAS Y CONCEPTOS UTILIZADOS

4.1 Kotlin

Kotlin es el lenguaje de programación oficial para Android. Características utilizadas:

Corrutinas (Coroutines)

```
CoroutineScope(Dispatchers.IO).launch {
    BDController.insertIngrediente(ingrediente)
}
```

4.2 Jetpack Compose (JPC)

Jetpack Compose es el toolkit moderno de Android para construir interfaces nativas con código declarativo.

Características:

- **Declarativo:** Describes QUÉ mostrar, no CÓMO mostrarlo
- **Recomposición:** La UI se actualiza automáticamente cuando cambia el estado
- **Sin XML:** Todo en Kotlin

4.3 Room Database

Room es la biblioteca de persistencia de Android que proporciona una capa de abstracción sobre SQLite.

Componentes de Room

1. Entities (Entidades)

```
@Entity(tableName = "ComponenteDieta")
data class ComponenteDieta(
    @PrimaryKey(autoGenerate = true) var id: Int = 0,
    var nombre: String = ""
)
```

Anotaciones:

- @Entity: Define una tabla
- @PrimaryKey: Clave primaria
- @ForeignKey: Relaciones entre tablas

2. DAOs (Data Access Objects)

@Dao

```
interface ComponenteRoomDAOI {
```

```
    @Insert
```

```
    suspend fun insertComponente(componente: ComponenteDieta): Long
```

```
    @Query("SELECT * FROM ComponenteDieta")
```

```
    suspend fun selectComponentes(): MutableList<ComponenteDieta>
```

```
    @Update
```

```
    suspend fun updateComponente(componente: ComponenteDieta)
```

```
    @Query("DELETE FROM ComponenteDieta WHERE id = :id")
```

```
    suspend fun deleteComponente(id: Int): Int
```

```
}
```

Relaciones en Room

Foreign Key (Clave Foránea):

```
@Entity(
```

```
    tableName = "Ingrediente",
```

```
    foreignKeys = [ForeignKey(
```

```
        entity = ComponenteDieta::class,
```

```
        parentColumns = ["id"],
```

```
        childColumns = ["componenteId"],
```

```
        onDelete = ForeignKey.CASCADE
```

```
    ]
```

```
)
```

- Un Ingrediente pertenece a un ComponenteDieta
- Si se elimina el ComponenteDieta, se eliminan sus Ingredientes (CASCADE)

Object Singleton para Control de BD

```
object BDController {
```

```
    lateinit var db: AppDataBase
```

```
    fun init(context: Context) {
```

```

        db = Room.databaseBuilder(
            context,
            AppDataBase::class.java,
            "appDB"
        ).build()
    }
}

```

Beneficios: Una sola instancia de la base de datos, acceso global y controlado

4.4 ViewModel y StateFlow

ViewModel

Componente que gestiona la lógica de UI y sobrevive a cambios de configuración:

```

class ViewModel_Listado : ViewModel() {
    private val _stateListado = MutableStateFlow(stateListado())
    val stateListado: StateFlow<stateListado> = _stateListado.asStateFlow()

    fun addComponenteDieta(componente: ComponenteDietaData) {
        _stateListado.update { currentState ->
            currentState.copy(
                listado = currentState.listado + componente
            )
        }
    }
}

```

4.5 Navigation Compose

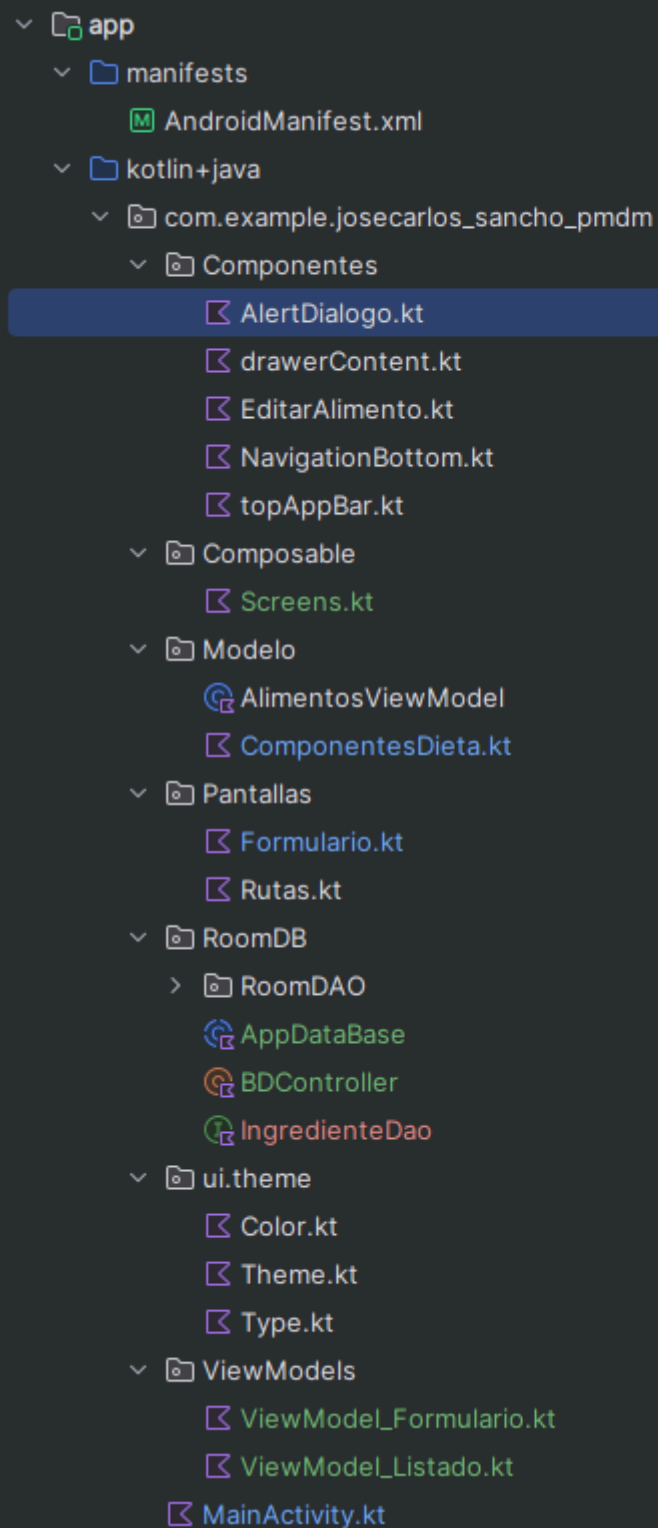
```

NavHost(navController = navController, startDestination = "main") {
    composable("main") { screenMain(...) }
    composable("formulario") { screenFormulario(...) }
    composable("listado") { screenListado(...) }
}

```

Conceptos:, NavController: Controlador de navegación, NavHost: Container de destinos

5. ESTRUCTURA DEL PROYECTO



The screenshot displays the project structure of an Android application in an IDE. The hierarchy is as follows:

- app
 - manifests
 - AndroidManifest.xml
 - kotlin+java
 - com.example.josecarlos_sancho_pmdm
 - Componentes
 - AlertDialogo.kt
 - drawerContent.kt
 - EditarAlimento.kt
 - NavigationBottom.kt
 - topAppBar.kt
 - Composable
 - Screens.kt
 - Modelo
 - AlimentosViewModel
 - ComponentesDieta.kt
 - Pantallas
 - Formulario.kt
 - Rutas.kt
 - RoomDB
 - RoomDAO
 - AppDataBase
 - BDController
 - IngredienteDao
 - ui.theme
 - Color.kt
 - Theme.kt
 - Type.kt
 - ViewModels
 - ViewModel_Formulario.kt
 - ViewModel_Listado.kt
 - MainActivity.kt

6. CAPA DE PERSISTENCIA (ROOM)

6.1 Configuración de la Base de Datos

AppDataBase.kt

```
@Database(entities = [ComponenteDieta::class, Ingrediente::class], version = 1)
abstract class AppDataBase: RoomDatabase() {
    abstract fun ingredienteDAO(): IngredienteRoomDAO
    abstract fun componenteDietaDAO(): ComponenteRoomDAO
}
```

6.2 BDController (Repository Pattern)

El **BDController** actúa como intermediario entre los ViewModels y la base de datos:

```
object BDController {
    lateinit var db: AppDataBase

    fun init(context: Context) {
        db = Room.databaseBuilder(context, AppDataBase::class.java, "appDB").build()
    }

    // COMPONENTES
    suspend fun insertComponente(componente: ComponenteDieta): Long {
        val dao = db.componenteDietaDAO()
        return dao.insertComponente(componente)
    }

    suspend fun selectAllComponentes(): MutableList<ComponenteDieta> {
        val dao = db.componenteDietaDAO()
        return dao.selectComponentes()
    }

    // ... más métodos
}
```

7. CAPA DE PRESENTACIÓN (VIEWMODELS)

7.1 ViewModel_Formulario

Gestiona el estado del formulario de creación

7.2 ViewModel_Listado

Gestiona la lista de componentes y el componente seleccionado:

8. INTERFAZ DE USUARIO (JETPACK COMPOSE)

8.1 Pantalla Formulario (screenFormulario)

Formulario

Listado

Nuevo Ingrediente

Nombre

Ingrediente

Valores Nutricionales

Proteínas (g)

0.0

Lípidos (g)

0.0

Hidratos (g)

0.0

Cantidad

1.0

Guardar

Formulario

Listado

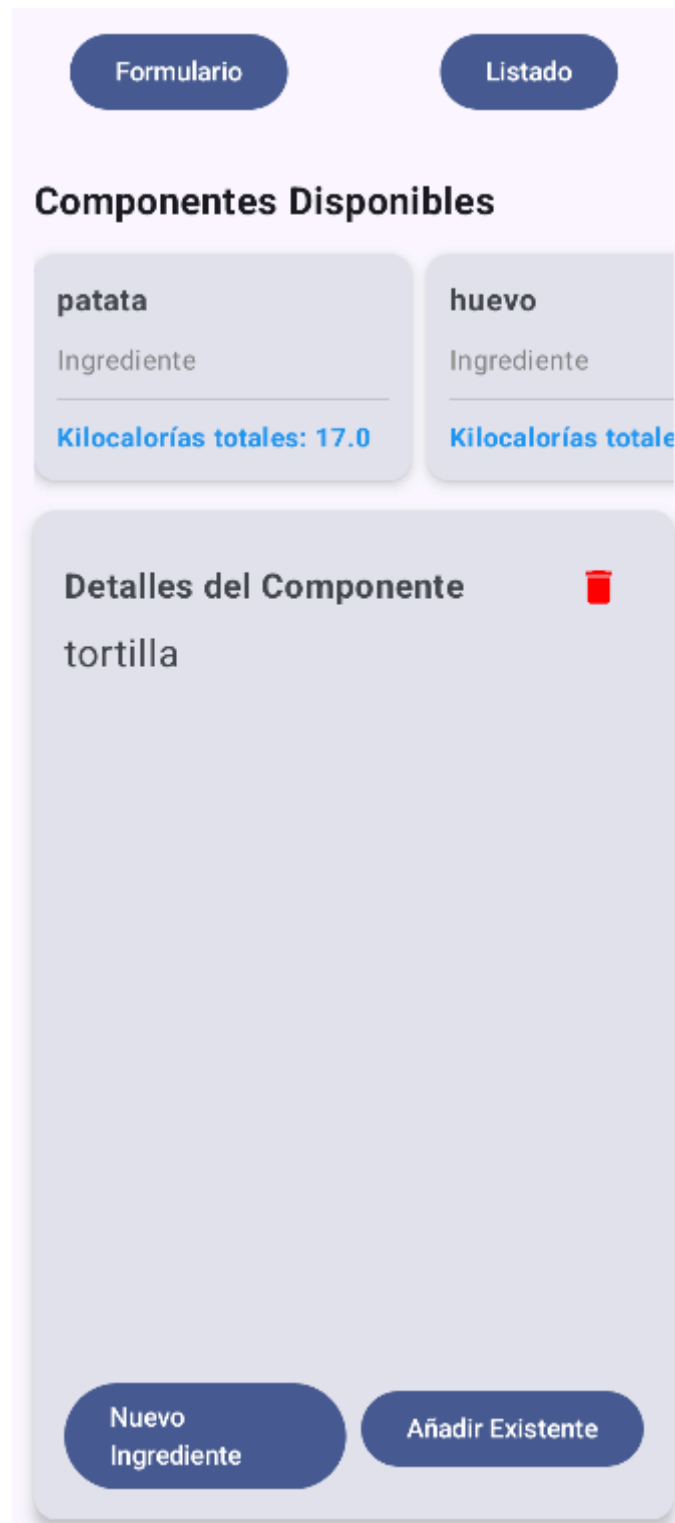
Nuevo Componente

Nombre

Seleccionar tipo

Guardar

8.2 Pantalla Listado (screenListado)



8.3 Añadir nuevo ingrediente o existente

Formulario

Listado

Componentes Disponibles

Nuevo Ingrediente

Nombre

Proteínas (g)

0.0

Lípidos (g)

0.0

Hidratos (g)

0.0

Cantidad

0.0

Cancelar

Añadir

Nuevo Ingrediente

Añadir Existente

Formulario

Listado

Componentes Disponibles

tortilla

Receta

Seleccionar Ingrediente

patata

HC: 1.0g Lip: 1.0g Pro: 1.0g

Calorías base: 17.0 kcal

huevo

HC: 0.0g Lip: 0.0g Pro: 0.0g

Calorías base: 0.0 kcal

Cancelar

Añadir

Nuevo Ingrediente

Añadir Existente



9. GUÍA DE USO

9.1 Crear un Ingrediente Básico

1. Navegar a Formulario

- Desde la pantalla principal, pulsar "Añadir Nuevo Alimento"
- O pulsar botón "Formulario" en la barra superior

2. Introducir datos

- Escribir nombre (ej: "Arroz")
- Seleccionar tipo "Ingrediente"
- Se mostrarán campos nutricionales

3. Valores nutricionales

- Proteínas: gramos por 100g (ej: 7.0)
- Lípidos: gramos por 100g (ej: 0.5)
- Hidratos: gramos por 100g (ej: 80.0)
- Cantidad: unidades (ej: 1.0 = 100g)

4. Guardar

- Pulsar "Guardar"
- Se crea el componente y su ingrediente asociado

9.2 Crear una Receta

1. Crear componente Receta

- En Formulario, introducir nombre (ej: "Paella")
- Seleccionar tipo "Receta"
- Guardar (se crea vacía)

2. Añadir ingredientes

- Ir a "Listado"
- Seleccionar la receta creada
- Pulsar "Nuevo Ingrediente" o "Añadir Existente"

3. Nuevo Ingrediente

- Introducir nombre y valores nutricionales
- Especificar cantidad
- Añadir

4. Añadir Existente

- Seleccionar ingrediente de la lista
- Modificar cantidad si es necesario
- Añadir

5. Resultado

- Las calorías se calculan automáticamente
- Se muestran en la card de la receta

10. CONCLUSIONES

Objetivos Cumplidos

Persistencia con Room Database

- Implementación completa de base de datos relacional
- Operaciones CRUD funcionales
- Relaciones entre entidades con integridad referencial

Arquitectura MVVM

- Separación clara de responsabilidades
- ViewModels gestionando lógica de negocio
- Estados reactivos con StateFlow

Jetpack Compose

- Interfaz moderna y declarativa
- Componentes reutilizables
- Navegación fluida

11. CONCLUSIÓN FINAL

Este proyecto representa una implementación completa y profesional de una aplicación Android moderna utilizando las mejores prácticas y tecnologías actuales.

Competencias demostradas:

- Dominio de Kotlin y sus características avanzadas
- Implementación de patrones arquitectónicos
- Gestión de bases de datos relacionales
- Creación de interfaces declarativas
- Programación reactiva y asíncrona
- Manejo de estados complejos
- Diseño de aplicaciones escalables

Desarrollado por: José Carlos Sancho Acosta

Fecha: Diciembre 2024

Institución: Proyecto de Desarrollo en Android - Segundo Trimestre

Esta documentación cubre todos los aspectos técnicos, arquitectónicos y funcionales del proyecto de Gestión de Alimentos con Room Database, implementado con Jetpack Compose y siguiendo las mejores prácticas de desarrollo Android.